

A LESSON IN RESEARCH METHODOLOGY

Proof Methods in Computer Science

Demonstration, Empirical, and Mathematical Argument

How a claim about a program, an algorithm, or a system is actually established — by building it, by measuring it, or by deriving it — and what each mode does and does not license.



Three Different Things Called “Proof”

Last session ended on Toulmin's Grounds — the evidence a claim rests on. Today's question is narrower and harder: what procedure actually manufactures admissible grounds in computing science?

- 1 In mathematics, “proof” means deductive certainty. In experimental science it means evidence strong enough to survive attempted refutation. A CS paper routinely uses the word in both senses, often on the same page.
- 2 Computing science can settle a claim three ways — by building the thing, by measuring the thing, or by deriving the thing from assumptions. Each licenses a different strength of conclusion.
- 3 The syllabus names them demonstration, empirical, and mathematical. They are instruments, not a ranking: a benchmark cannot establish necessity, and a theorem cannot establish that real users are faster.
- 4 Choosing the wrong instrument is not a stylistic slip. It makes the claim unprovable in principle — however good the underlying work is.

THE CATEGORY ERROR

You cannot benchmark your way to a universal claim, and you cannot prove your way to a claim about real workloads.

Reviewers reject far more papers for a mismatch between the claim and the mode of proof than for arithmetic errors in the evidence itself.

Match the instrument to the quantifier in your claim.

The Three Modes of Proof

Each mode answers a different question, produces a different artifact, and supports a different logical form of claim.

MODE 1

Demonstration

“It can be done.”

ESTABLISHES

Feasibility and existence — that a design is realizable.

INSTRUMENT

A working artifact, exercised under the conditions that made the claim doubtful.

LOGICAL FORM

$\exists$ a system S such that property P holds of S .

HOME FIELDS

Systems, PL, HCI, security, architecture.

MODE 2

Empirical

“It behaves this way, under these conditions.”

ESTABLISHES

Regularity in observed behaviour; effect size; comparison.

INSTRUMENT

Controlled measurement over a sample, plus statistical inference.

LOGICAL FORM

Under conditions C , X outperforms Y by δ , with confidence c .

HOME FIELDS

ML, databases, SE, networking, performance analysis.

MODE 3

Mathematical

“It must be so — and here is why.”

ESTABLISHES

Necessity within a stated model; bounds; impossibility.

INSTRUMENT

Deductive derivation from explicit assumptions.

LOGICAL FORM

$\forall$ inputs of class I , property P holds — no exceptions.

HOME FIELDS

Algorithms, complexity, verification, cryptography.

The three are not a hierarchy — they answer different questions. Ranking evidence is Shaw's taxonomy (Lecture 1); matching evidence to a claim is your job as an author.

What Each Mode Can — and Cannot — Establish

Read the third column as the reviewer's opening question, and the fourth as the way papers most often fail it.

MODE	ESTABLISHES	CANNOT ESTABLISH	CHARACTERISTIC FAILURE
Demonstration	Existence and feasibility. A constructive proof that the design is realizable and its constraints are jointly satisfiable.	Generality, comparative superiority, or any performance guarantee beyond the configuration actually exercised.	Reporting one working configuration as though it were a universal property of the approach.
Empirical	Behaviour over a sampled population: effect size, comparative performance, variance, and the conditions each holds under.	Necessity, and behaviour outside the sampled distribution of workloads, datasets, hardware, or users.	Generalising past the benchmark population — or comparing against a baseline nobody tuned.
Mathematical	Necessity inside the stated model: worst-case bounds, correctness, security reductions, impossibility results.	That the model matches deployment. Constants, cache behaviour, and specification errors sit outside the proof.	A theorem about a machine model no real system inhabits, with the load-bearing premise stated once and never revisited.

Every claim carries a quantifier — “there exists,” “on average under C,” “for all.” The quantifier tells you which row you are in.

PART 1 → PART 2

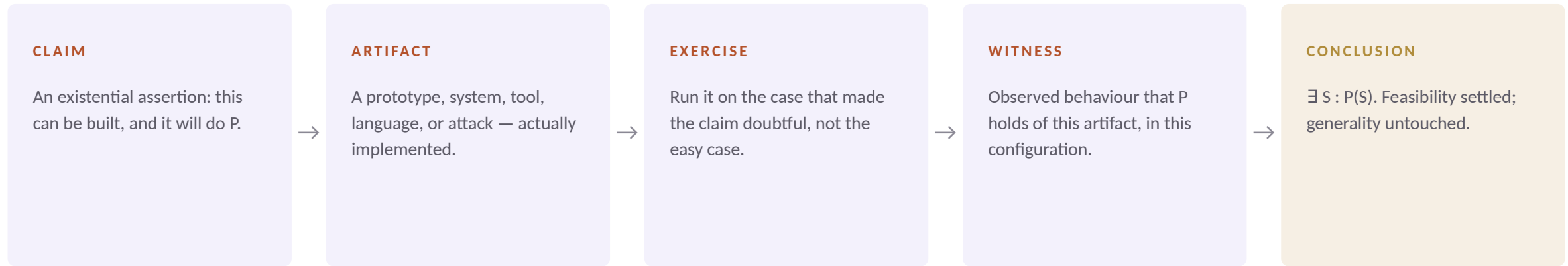
From a taxonomy of proof to the first instrument: building it

If a claim can be settled by exhibiting a working thing, what exactly has been settled — and what has quietly not been?

Next: proof by demonstration — the constructive existence proof of engineering.

Proof by Demonstration

Establishing a claim by constructing an artifact and exercising it under conditions that make the claimed behaviour directly observable.



The engineer's existence proof

Where a mathematician writes “such an object exists,” a systems researcher builds one and switches it on. Both are constructive arguments; only the medium differs.

Strongest for “can”, weakest for “is”

A demonstration is the most direct answer to “is this possible?” and the least informative answer to “is this generally true?”

Why whole fields rely on it

In systems, HCI, PL and security, what is theoretically possible and what is practically achievable diverge sharply. The artifact is the only witness that closes that gap.

What a Demonstration Proves — and What It Doesn't

The same artifact, read as evidence for two different claims, is decisive for one and silent on the other.

IT DOES PROVE

- Realizability — the design is not merely a diagram; the constraints can be met simultaneously.
- That an assumed barrier is not fundamental — “this was thought impractical” is refuted by a running counter-instance.
- An upper bound on difficulty — the problem is at most as hard as the effort this artifact took.
- Existence of a design point — a concrete location in the design space that others can now measure against.

IT DOES NOT PROVE

- Superiority — “better than X” is a comparative claim, and it silently switches you into the empirical mode.
- Generality — behaviour outside the demonstrated workload, scale, dataset, or hardware is simply unobserved.
- Optimality or necessity — that this is the best way, or the only way, is a mathematical claim.
- Robustness — that it survives adversarial, degraded, or long-running conditions you never exercised.

A demonstration answers “is it possible?” Every “is it better?” sentence in your paper has quietly changed instruments — and now owes a baseline.

Making a Demonstration Admissible

Six moves that separate Shaw's "Analysis" from Shaw's "Example" — the difference between evidence and an anecdote.

1 State the existential claim

Name the property the artifact is meant to witness, in one sentence, before any implementation detail. If you cannot write it, the demonstration has no target.

2 Report the configuration fully

Versions, hardware, dataset, parameters, seeds. An unreproducible demonstration is not admissible evidence — it is a story about a machine nobody else has.

3 Ship the artifact

Code, data, and run scripts. Artifact-evaluation badges at ICSE, PLDI, OSDI and SOSP exist precisely because a demonstration is only checkable when it is re-runnable.

4 Exercise the hard case

The demonstration must cover the condition that made the claim doubtful. A prototype shown only on the easy input witnesses nothing that was ever in question.

5 Ablate the novel component

Remove each new mechanism and show the property degrades or fails. Without ablation you have shown that a system works, not that your idea is what made it work.

6 Name the boundary

State the configuration beyond which you did not demonstrate anything. This is Toulmin's rebuttal component, doing its job in advance.

Worked Example: A Demonstration, Read as an Argument

Claim under test: “A general-purpose programming model can let non-specialists write fault-tolerant computations over thousands of commodity machines.” — *Dean & Ghemawat, MapReduce, OSDI 2004*

TOULMIN COMPONENT	HOW THE DEMONSTRATION SUPPLIES IT
Claim	The programming model is feasible and adoptable at production scale — an existential, not a comparative, assertion.
Grounds	A working implementation, plus roughly a thousand distinct jobs written in the model and run inside Google over the reporting period.
Warrant	If the model is realizable and non-specialists actually express real jobs in it at this scale, then the feasibility claim is settled.
Backing	Established functional-programming primitives (map, reduce) and an existing distributed filesystem substrate the design could rest on.
Qualifier	Holds for batch, data-parallel workloads on large commodity clusters — the scope the deployment actually covers.
Rebuttal	Says nothing about iterative or low-latency workloads — precisely the gap that Spark, Dryad and their successors were later built to attack.

Notice what is never claimed: no optimality, no proof, no head-to-head benchmark against alternatives that did not yet exist. The paper's power comes from a correctly bounded claim.

Empirical Proof: The Inductive Leap

Establishing a claim by systematic observation of instances, then inferring a regularity that extends beyond the instances observed.

OBSERVED SAMPLE

The runs you actually made: these traces, this hardware, these seeds, these participants.

induction



the warrant

UNOBSERVED POPULATION

Every workload, machine, dataset and user your claim silently ranges over.

THE DISTINCTION THAT MATTERS

A benchmark result is a measurement.

A benchmark claim is an inference.

Reviewers attack the second.

This is why an empirical claim with no stated population is not merely imprecise — it is unfalsifiable. Nothing could count against a claim that never said what it was about.

- **Every result is a sample; every claim is about a population.**

The measurement is not the contested part. The inference from the sample to the population is — and that inference is exactly Toulmin's warrant.

- **The warrant is never free.**

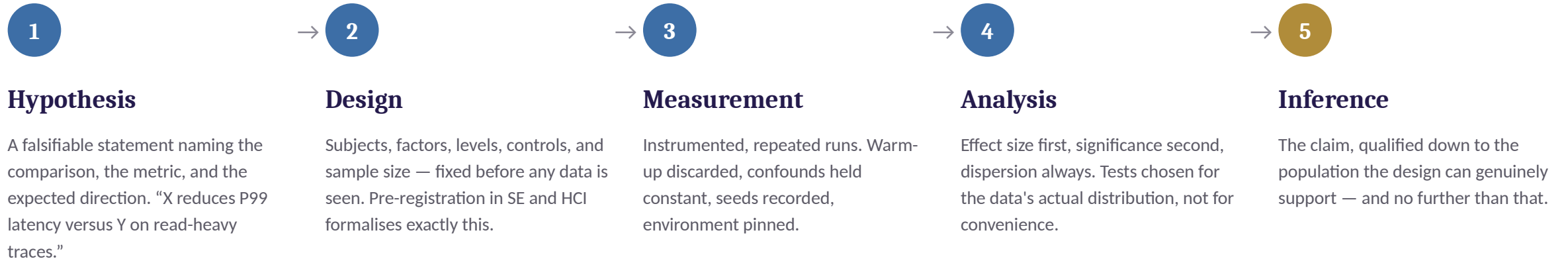
It is bought with sampling design, controls, repetition and statistics. Skip the purchase and the leap is unlicensed, however carefully the numbers were collected.

- **Hume's problem, in modern dress.**

No finite set of benchmark runs entails the next one. Empirical proof buys graded confidence, never necessity — which is why its claims must carry qualifiers.

Anatomy of an Empirical Study

Five stages. The order is not a convention — it is what makes the result falsifiable at all.



WHY THE ORDER IS LOAD-BEARING

Stages 1 and 2 must precede stage 3. Fix the hypothesis after seeing the data and every subsequent p-value is meaningless: the analysis is now guaranteed to confirm whatever the data happened to show. Reordering these stages is the mechanism behind HARKing, p-hacking, and the replication crisis — and it silently converts an empirical study into an elaborate description of noise.

Benchmarks, Baselines, and Controls

Three design decisions that determine whether measurements are admissible at all — made before the first run, not defended after the last.

THE POPULATION, MADE CONCRETE

Benchmark

The workload standing in for everything your claim ranges over. It must be representative of that population, publicly available so others can re-run it, and chosen before you see which one flatters your method.

Risk: a whole subfield over-fitting to one dataset, until progress on the benchmark stops tracking progress on the problem.

THE ALTERNATIVE, AT FULL STRENGTH

Baseline

The strongest reasonable competitor, tuned with the same care and the same effort budget as your own system. Use the current state of the art, not the version that was current when you started.

Risk: an untuned or outdated baseline turns a comparison into a strawman, and the result evaporates the moment someone tunes it.

EVERYTHING ELSE, HELD STILL

Control

All variables except the factor under study, fixed: hardware and thermal state, compiler flags, warm-up, dataset order, random seeds, background load, and library versions.

Risk: an uncontrolled confound means the difference you measured may have nothing to do with the change you made.

The most common objection in empirical CS review is not “your numbers are wrong.” It is “your baseline was weak,” or “your benchmark was chosen after the fact.”

Statistics: The Warrant Made Explicit

Statistical reporting is not decoration on a result. It is the written form of the reason your evidence supports your claim.

WEAK

“Our method is 12% faster than the baseline.”

One run per configuration, no dispersion, no test, no effect size. Nothing in the sentence distinguishes a real improvement from ordinary run-to-run variance on a shared machine — and timing noise on modern hardware routinely exceeds 12%.

The warrant is missing entirely. The reader is asked to make the inductive leap on trust.

STRONG

“Across 30 runs per configuration on 3 hardware generations, median speed-up 1.12× (IQR 1.08–1.17); Mann–Whitney U, $p < 0.01$; Cliff's $\delta = 0.61$ (large).”

Sample size, dispersion, a test appropriate to skewed timing data, and a magnitude the reader can weigh against engineering cost. The population is named, and the qualifier is visible in the sentence itself.

The warrant is stated, backed, and bounded — Toulmin's components, in statistical clothing.

Report distributions, not points

Execution times are skewed and rarely normal. Give median and spread; a mean alone hides the tail your users will actually feel.

Effect size, then significance

A p-value answers “distinguishable from noise?”. Effect size answers “does it matter?”. Only the second is a research finding.

Fix and publish your seeds

An unreported random seed is an uncontrolled variable. Reporting the best of many seeds is a selection effect, not a result.

Correct for multiple comparisons

Test twenty configurations at $\alpha = 0.05$ and one will look significant by construction. Adjust, or pre-register the one you meant.

Threats to Validity

Four standing questions any empirical claim must answer. Each maps to a different link in the chain from measurement to conclusion.

THREAT 1

Internal validity

Did the factor you changed actually cause the difference you measured?

TYPICAL THREATS

Warm-up and JIT effects • thermal throttling • hardware drift between runs • dataset ordering • a change shipped alongside yours.

THREAT 2

External validity

Does the result extend beyond the workloads, machines, and users you sampled?

TYPICAL THREATS

A single benchmark suite • one hardware generation • one dataset scale • students standing in for professional developers.

THREAT 3

Construct validity

Does your metric measure the property your claim is actually about?

TYPICAL THREATS

Throughput as a proxy for user-perceived latency • raw accuracy on an imbalanced set • lines of code as a proxy for effort.

THREAT 4

Conclusion validity

Do the statistics support the strength of the claim you drew from them?

TYPICAL THREATS

Too few runs • no reported variance • violated test assumptions • p-hacking • HARKing • fishing across many configurations.

In Toulmin's terms, a threats-to-validity section is the paper naming its own rebuttals before a reviewer gets to name them — on the author's terms rather than the reviewer's.

Mathematical Proof: Deductive Certainty

Establishing a claim by deriving it from explicitly stated assumptions, using truth-preserving rules of inference.

- **What it buys you: universality.**

A theorem holds for every input in its scope — including the inputs nobody has run, will run, or could afford to run. No amount of measurement reaches that far.

- **What it costs you: the model.**

The conclusion is only as true as its premises. The RAM cost model, uniform memory access, an adversary's bounded power, an independence assumption — each is a premise smuggled in with the notation.

- **What only it can do: impossibility.**

No experiment shows that something cannot be done; a failed search is not a proof of absence. Impossibility results are mathematics' unique contribution to engineering.

A CS claim is “mathematically proved” when it follows necessarily from stated assumptions — not when it is merely expressed in symbols. Notation is not proof.

WHERE THE CONTENT ACTUALLY SITS

A proof transfers certainty from premises to conclusion. It does not create certainty.

So the real content of any theorem lives in its assumptions. When you read a proof critically, you are rarely hunting for an invalid inference step — you are hunting for the premise that quietly did all the work, and asking whether any real system satisfies it.

The Proof-Technique Toolkit

Eight techniques that cover most of what a CS paper or thesis needs. The skill is not executing them — it is recognising which one the claim calls for.

Direct proof

Chain the assumptions forward to the conclusion.

Correctness of a straight-line transformation; closure properties.

Contraposition

Prove $\neg Q \Rightarrow \neg P$ instead of $P \Rightarrow Q$.

When the negated statement is the tractable one to reason about.

Contradiction

Assume $\neg P$ and derive an absurdity.

Non-existence claims; minimal-counterexample arguments.

Induction

Base case plus inductive step, over $\mathbb{N}$ or over structure.

Recursion, recursive data types, algorithm correctness in n .

Loop invariant

Initialisation, maintenance, termination.

Correctness of iterative algorithms — induction, specialised to loops.

Construction

Exhibit the object the claim says exists.

Achievability results and upper bounds; the theorist's demonstration.

Counter-example

One instance refutes a universal claim.

Killing a conjecture, or a guarantee someone stated too broadly.

Reduction

Transform problem A into problem B.

NP-hardness, undecidability, lower bounds, cryptographic security.

Also in regular use: exhaustive case analysis, the probabilistic method, adversary arguments, and diagonalisation — the last of which we meet again three slides from now.

Worked Example: Correctness by Loop Invariant

Claim under test: “Insertion sort terminates, and on termination the array is a sorted permutation of its input.”

```
for j = 2 to A.length
  key = A[j]
  i = j - 1
  while i > 0 and A[i] > key
    A[i+1] = A[i]
    i = i - 1
  A[i+1] = key
```

THE INVARIANT

At the start of each iteration of the for loop, $A[1 \dots j-1]$ contains exactly the elements originally in $A[1 \dots j-1]$, in sorted order.

1

Initialisation

Before the first iteration $j = 2$, so $A[1 \dots 1]$ is a single element — trivially sorted, and trivially a permutation of itself. The invariant holds on entry.

2

Maintenance

The while loop shifts every element of $A[1 \dots j-1]$ greater than key one position right, then places key in the vacated slot. Only positions change, so the multiset is preserved; key lands after all smaller elements and before all larger ones, so $A[1 \dots j]$ is sorted when j is incremented.

3

Termination

The for loop ends with $j = n + 1$. Substituting into the invariant gives: $A[1 \dots n]$ contains the original elements, in sorted order — which is exactly the postcondition, so insertion sort is correct.

Termination of the inner loop needs its own argument: i strictly decreases on each pass and is bounded below by 0, so the while loop cannot run forever. A correctness proof that skips this step establishes only partial correctness.

Worked Example: A Lower Bound No Benchmark Could Give

Claim under test: “Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.”

1

Model

Represent any comparison sort as a binary decision tree. Internal nodes are comparisons, the two children are the two outcomes, and each leaf is one permutation the algorithm can output.

2

Count the leaves

To sort correctly, the algorithm must be able to produce every one of the $n!$ possible orderings. So the tree has at least $n!$ reachable leaves.

3

Bound the height

A binary tree of height h has at most 2^h leaves. Therefore $2^h \geq n!$, and so $h \geq \log_2(n!)$.

4

Estimate

By Stirling's approximation, $\log_2(n!) = \Theta(n \log n)$. The longest root-to-leaf path is the worst-case comparison count, so that count is $\Omega(n \log n)$ — as claimed.

WHAT THIS BUYS

A statement about every comparison sort that will ever be written, including ones nobody has thought of yet. No quantity of benchmarking could establish this — measurement can only ever report on algorithms that already exist.

WHAT IT COSTS

Notice the premise doing all the work: comparison-based. Counting sort and radix sort run in linear time precisely because they leave that model. The theorem is airtight and, for the right input, entirely escapable.

Reduction and Impossibility

The two mathematical results that most directly change what engineers do next — by telling them which doors are locked.

TECHNIQUE

Reduction for hardness

If a known-hard problem B can be transformed into your problem A in polynomial time, then A is at least as hard as B. Cook-Levin gave the first NP-complete problem; every later NP-completeness result is a chain of reductions back to it.

Consequence for practice: stop searching for an efficient exact algorithm. Redirect the effort to approximation, heuristics, parameterised algorithms, or restricted input classes.

TECHNIQUE

Diagonalisation for undecidability

Turing's halting argument constructs a program that contradicts any supposed universal decider. Rice's theorem generalises it: every non-trivial semantic property of programs is undecidable.

Consequence for practice: static analysers and verifiers must be approximate by necessity. Soundness and completeness cannot both be had, so every tool chooses which to sacrifice — and must say which.

RESULT CLASS

Impossibility results

FLP (1985) proves that no deterministic protocol solves consensus in an asynchronous system if even one process may crash. Cryptographic lower bounds and distributed-systems trade-offs work the same way.

Consequence for practice: real systems adopt randomisation, partial synchrony, or failure detectors — engineered escapes from a premise, not refutations of the theorem.

An impossibility proof is a contribution because it redirects a field's effort. It does not tell you what to build — it tells you to stop pushing on a door that is locked, and go find the window.

Where Proof Meets Practice

Four places where the mathematical mode and the empirical mode disagree — and why the disagreement is information, not a defect.

THE TENSION	WHAT ACTUALLY HAPPENS	WHAT AN HONEST PAPER DOES
Asymptotics hide constants	$\Theta(n \log n)$ with a constant of 10^6 loses to $\Theta(n^2)$ at every input size you will ever deploy. Complexity classes rank growth, not running time.	Report the crossover point, or say plainly that the claim is asymptotic only.
The cost model is a premise	The RAM model prices every memory access identically. Cache hierarchies, branch prediction, NUMA and prefetching do not — and the gap is orders of magnitude.	State the model. If it is violated in deployment, measure the violation instead of ignoring it.
Worst case versus observed case	Simplex is exponential in the worst case and fast in practice; SAT is NP-complete, and solvers routinely dispatch instances with millions of variables.	Pair the bound with a distributional or empirical claim about the inputs that actually occur.
Verified is not the same as correct	A proof is about a model of the program. Bugs then live in the gap: the specification, the compiler, the hardware, or an assumption the deployment quietly violates.	Name what was verified, and against which specification — CompCert and seL4 both do this carefully.

None of this is an argument against proof. It is an argument for saying explicitly which mode carries which part of your claim — and where you handed off from one to the other.

Triangulation: Matching Claim to Proof Mode

Find the sentence carrying your contribution, read its quantifier, then read across.

THE CLAIM YOU ARE MAKING	PRIMARY MODE	SUPPORTING MODE	WHAT A REVIEWER WILL ASK FIRST
“This is possible.”	Demonstration	—	Is the artifact available, re-runnable, and exercised on the hard case?
“This is faster / more accurate.”	Empirical	Demonstration	Was the baseline tuned, and was the benchmark fixed before you saw results?
“This always holds.”	Mathematical	Empirical	Do your assumptions hold in any deployment anyone actually runs?
“This cannot be done.”	Mathematical	—	Is the model general enough for the impossibility to actually bite?
“This is usable / adoptable.”	Empirical (human)	Demonstration	Enough participants — and is the construct you measured the one you claim?

Strong papers stack modes: prove the bound, build the system, then measure the gap between the two — Shaw's Analysis and Evaluation, reinforcing each other.

Failures of Proof to Watch For

Lecture 1 catalogued argumentative fallacies. These are their specifically evidential cousins — each one a mode used to support a claim it cannot reach.

Demonstration as generality

One working configuration reported as though it were a property of the approach. The existential quantifier silently becomes a universal one somewhere between the results section and the abstract.

The strawman baseline

A comparison against an untuned, outdated, or deliberately naive alternative. The result is real; it just measures the baseline's neglect rather than your contribution.

Asymptotic sleight of hand

Claiming practical superiority from a complexity class, with constants and lower-order terms left unmentioned — and no measurement offered at the sizes anyone runs.

The seed lottery

Reporting the best random seed, the best run, or the best checkpoint. This is selection, not measurement, and it is invisible unless variance is reported.

The hidden premise

A theorem whose real work is done by an assumption stated once in a preamble and never revisited — independence, synchrony, bounded adversaries, uniform cost.

HARKing

Hypothesising after the results are known: the finding is chosen from the data, then presented as though it had been predicted. The empirical twin of the post-hoc warrant.

A Checklist for Your Own Proofs

- ✓ **Name the mode.** In one sentence: is this claim settled by construction, by measurement, or by derivation? If the answer is “all three, a bit,” none of them is finished.
- ✓ **Match the mode to the quantifier.** “There exists” → demonstration. “On average, under conditions C” → empirical. “For all inputs of class I” → mathematical. A mismatch here is not fixable by better writing.
- ✓ **Make the assumptions visible.** Every theorem's premises, every study's population, every demonstration's configuration — stated where a reader will find them, not buried in a footnote.
- ✓ **Strengthen the alternative before you publish.** Tune the baseline. Hunt for the counter-example. Attack your own model. Whatever you skip here, a reviewer will do for you, less charitably.
- ✓ **Report dispersion, not just the centre.** Runs, variance, effect size, confidence. If you cannot supply them, say plainly that you have an existence proof rather than a comparison.
- ✓ **Say where it stops.** The workload, the scale, the assumption, the input class beyond which you proved, measured, and built nothing at all.

Discussion

In pairs, ten minutes. Bring one answer back for class critique.

1

Take a paper from your own area. Which of the three modes carries its central claim — and does the quantifier in the abstract match the mode used in the evaluation?

2

Find an empirical CS paper whose headline claim is stated universally (“X is faster than Y”). Rewrite it with the qualifier its evidence actually supports. How much of the claim survives?

3

Pick an impossibility result you know — FLP, the halting problem, the $\Omega(n \log n)$ sorting bound. Which premise would you have to break to escape it, and is that premise breakable in a real system?

4

When is a demonstration sufficient on its own? Name a subfield where “we built it and it runs” is a complete contribution, and defend that position against someone who wants a benchmark.

Key Takeaways

- 1 Computing science settles claims three ways — by demonstration, by empirical study, and by mathematical derivation — and each licenses a different strength of conclusion.
- 2 A demonstration settles existence and feasibility. It never settles generality or superiority; the moment your sentence turns comparative, you owe a baseline.
- 3 Empirical proof is an inference from a sample to a population. Its warrant is bought with design, control, repetition and statistics — and reported as effect size and dispersion, not significance alone.
- 4 Mathematical proof gives universality inside a model, and is the only mode that can establish impossibility — but it inherits the truth of its assumptions, which is where critical reading should aim.
- 5 The category error to avoid is a mismatch between the claim's quantifier and the mode used to support it. The mark of strong work is stacking modes, and saying honestly where each one stops.

Further Reading

Cormen, Leiserson, Rivest & Stein. Introduction to Algorithms. 4th ed., MIT Press, 2022. — loop invariants; the decision-tree lower bound.

Velleman, D. J. How to Prove It: A Structured Approach. 3rd ed., Cambridge University Press, 2019.

Sipser, M. Introduction to the Theory of Computation. 3rd ed., Cengage, 2013. — diagonalisation, reduction, undecidability.

Garey, M. R. & Johnson, D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. Freeman, 1979.

Fischer, M. J., Lynch, N. A. & Paterson, M. S. “Impossibility of Distributed Consensus with One Faulty Process.” Journal of the ACM, 1985.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B. & Wesslén, A. Experimentation in Software Engineering. Springer, 2012.

Jain, R. The Art of Computer Systems Performance Analysis. Wiley, 1991.

Shaw, M. “Writing Good Software Engineering Research Papers.” Proc. ICSE, 2003. — the validation-type taxonomy from Lecture 1.

Dean, J. & Ghemawat, S. “MapReduce: Simplified Data Processing on Large Clusters.” Proc. OSDI, 2004.